

Microsoft

Building Intelligent Troubleshooting Agents

Module 1: LLM fine-tuning for task-specific adaptation

Module 2: Fundamentals of AI agents

Module 3: Natural language processing for troubleshooting

Module 4: Implementing the troubleshooting agent

Module 5: Testing and optimizing the agent

Designing test cases
Video • 8 min

Exploration of test case design
Reading • 10 min

Practice activity: Designing test cases for ML systems
Practice Assignment • 10 min

Reflection: Designing test cases for ML systems
Practice Assignment • 10 min

Walkthrough: Designing test cases for ML systems (Optional)
Video • 5 min

Hear from an expert: Accounting for cultural, language, and contextual nuances
Video • 4 min

Optimizing response time and accuracy
Reading • 10 min

Explanation of optimization techniques
Video • 8 min

Practice activity: Implementing optimization techniques
Reading • 10 min

Reflection: Implementing optimization techniques
Practice Assignment • 10 min

Walkthrough: Implementing optimization techniques (Optional)
Video • 7 min

Evaluating agent effectiveness
Reading • 10 min

Practice activity: Evaluating agent effectiveness
Reading • 10 min

Reflection: Evaluating agent effectiveness
Practice Assignment • 10 min

Walkthrough: Evaluating agent effectiveness (Optional)
Video • 5 min

Hear from an expert: Designing with the end user in mind
Video • 2 min

Summary: Testing and optimizing the agent
Video • 5 min

Practice activity: Testing and optimizing the agent
Reading • 10 min

Reflection: Testing and optimizing the agent
Practice Assignment • 10 min

Walkthrough: Testing and optimizing the ML agent (Optional)
Video • 7 min

Graded quiz: Testing and optimizing the agent
Graded Assignment • 10 min

Hear from an expert: Resolving unexpected issues during implementation
Video • 5 min

Course summary
Video • 8 min

Course assignment: Producing a troubleshooting agent
Reading • 10 min

Reflection: Producing a troubleshooting agent
Practice Assignment • 10 min

Walkthrough: Producing a troubleshooting agent (Optional)
Video • 4 min

Course assignment: Drafting the technical report
Peer graded Assignment • 10-30m

Course assignment: Drafting the technical report
Review Your Draft

Congratulations on completing the course!
Video • 3 min

Practice activity: Implementing optimization techniques

Introduction

In this activity, you will apply key optimization techniques to a machine learning (ML) model to improve its response time and accuracy. By the end of this activity, you'll have hands-on experience with techniques such as model pruning, quantization, and hyperparameter tuning. These methods are designed to help you optimize your model for real-world applications where speed and precision are essential.

By the end of this activity, you will be able to:

- Apply model pruning to reduce model complexity and improve performance.
- Explain how quantization helps in speeding up inference.

Step-by-step process to optimize ML models

This reading will guide you through the following steps:

- Step 1: Set up the environment.
- Step 2: Apply model pruning.
- Step 3: Apply model quantization.
- Step 4: Perform hyperparameter tuning.
- Step 5: Evaluate the trade-offs.

Step 1: Set up the environment

Instructions

Create a new Jupyter notebook. Start by setting up your Python 3.8 Azure ML kernel. Ensure you have the necessary libraries installed, including Scikit-Learn, TensorFlow, or PyTorch, depending on the model you're working with.

Setup commands

```
1 !import subprocess
2
3 # Install scikit-learn and tensorflow using pip
4 !pip install scikit-learn tensorflow
5
6 # Install tensorflow-model-optimization
7 !pip install tensorflow-model-optimization
8
9 # Warning! This runs with an 8-bit kernel by default...
10 !conda install tf.keras
11
12 # Need tf.keras for tf-model-optimization to work
```

Explanation

This setup ensures that you have the right tools to apply optimization techniques such as pruning and quantization.

Step 2: Apply model pruning

Begin by applying model pruning to reduce the size of your model. In this step, you'll remove unnecessary neurons or branches that have minimal impact on the overall accuracy.

Instructions

- Train the model on a dataset.
- Apply pruning by removing neurons or branches with the least contribution to model performance.

Example (TensorFlow)

```
1 import tensorflow_model_optimization as tfmot
2 import tensorflow as tf
3 from tensorflow.keras.datasets import mnist
4
5 # Load dataset
6 (x_train, y_train), (x_test, y_test) = mnist.load_data()
7 x_train, y_train = x_train / 255.0, y_train / 255.0
8
9 # Build a simple model
10 model = tf.keras.models.Sequential([
11     tf.keras.layers.Flatten(input_shape=(28, 28)),
12     tf.keras.layers.Dense(128, activation='relu'),
13     tf.keras.layers.Dropout(0.2),
14     tf.keras.layers.Dense(10, activation='softmax')
15 ])
16
17 # Compile the model
18 model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
19
20 # Train the model
21 model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
22
23 # Apply pruning to the model
24 pruning_params = {
25     'pruning_scope': tfmot.sparsity.keras.PolyomialDecay(initial_sparsity=0.6, final_sparsity=0.3, begin_step=0, end_step=1000)
26 }
27 pruned_model = tfmot.sparsity.keras.prune_low_magnitude(model, **pruning_params)
28
29 # Compile the pruned model
30 pruned_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
31
32 # Train the pruned model to finalize pruning
33 callbacks = [tfmot.sparsity.keras.callbacks.ReduceRedundancy()]
34 pruned_model.fit(x_train, y_train, epochs=2, validation_data=(x_test, y_test), callbacks=callbacks)
35
36 # Strip pruning wrappers to remove pruning-specific layers and metadata
37 pruned_model = tfmot.sparsity.keras.strip_pruning(pruned_model)
```

Explanation

Pruning reduces the size of the model and the computational resources required during inference without sacrificing much accuracy. This is especially useful for deploying models on resource-constrained devices.

Step 3: Apply model quantization

Next, apply quantization to your model. This will reduce the precision of the model's parameters, speeding up inference, especially on devices with limited computational resources.

Instructions

- Quantize the model's weights from 32-bit floating-point to 8-bit integers.
- Evaluate the model's performance after quantization to ensure that accuracy remains acceptable.

Example (TensorFlow Lite quantization)

```
1 # Convert the pruned model to a TensorFlow Lite quantized model
2 converter = tf.lite.TFLiteConverter.from_keras_model(pruned_model)
3 converter.optimizations = [tf.lite.Optimize.DEFAULT]
4 quantized_model = converter.convert()
```

Explanation

Quantization reduces the size of the model and speeds up inference by lowering the precision of its weights, which is particularly useful for deployment on mobile devices or edge devices.

Step 4: Perform hyperparameter tuning

Perform hyperparameter tuning to find the optimal combination of parameters that maximize accuracy while maintaining reasonable response times.

Instructions

- Use a grid search or a randomized search to explore different hyperparameter combinations.
- Choose the parameters that offer the best balance between model performance and computational cost.

Example (grid search in Scikit-Learn)

```
1 # Measure accuracy of the quantized model using the test set
2 interpreter = tf.lite.Interpreter(model_content=quantized_model)
3 interpreter.allocate_tensors()
4
5 input_index = interpreter.get_input_details()[0]['index']
6 output_index = interpreter.get_output_details()[0]['index']
7
8 # Evaluate accuracy
9 correct_predictions = 0
10 for i in range(len(x_test)):
11     input_data = x_test[i:i+1].astype('float32')
12     interpreter.set_tensor(input_index, input_data)
13     interpreter.invoke()
14     output = interpreter.get_tensor(output_index)
15     predicted_label = output.argmax()
16     if predicted_label == y_test[i]:
17         correct_predictions += 1
18
19 accuracy = correct_predictions / len(x_test)
20 print(f'Quantized model accuracy: {accuracy * 100:.2f}%')
```

Explanation

Hyperparameter tuning helps you find the best settings for your model, improving accuracy while minimizing computational requirements.

Step 5: Evaluate the trade-offs

Finally, evaluate the trade-offs between model complexity, accuracy, and response time. Document the changes in performance (e.g., accuracy and latency) before and after applying these optimization techniques.

Instructions

- Compare the original model's accuracy and response time with the optimized model.
- Identify the key trade-offs. (For example, did pruning reduce accuracy but speed up inference significantly?)
- Document your findings, and decide which optimizations are most suitable for your use case.

Explanation

Understanding the trade-offs between accuracy and speed helps you make informed decisions about which optimization to apply in production environments.

Conclusion

In this activity, you applied several optimization techniques—pruning, quantization, and hyperparameter tuning—to improve the performance of your ML model. By balancing accuracy with response time, you can ensure that your model is not only accurate but also efficient and ready for real-world applications.

Mark as completed

coach

Hi, Venkatesh!
How can I help?

Explain this topic in simple terms

Give me a summary

Give me practice questions

Give me real-life examples

Ask me anything

Go to next item

Coach is powered by AI, so check for mistakes and don't share sensitive info. Your data will be used in accordance with OpenAI's Privacy Notice